Project Design Phase-II
Technology Stack (Architecture & Stack)

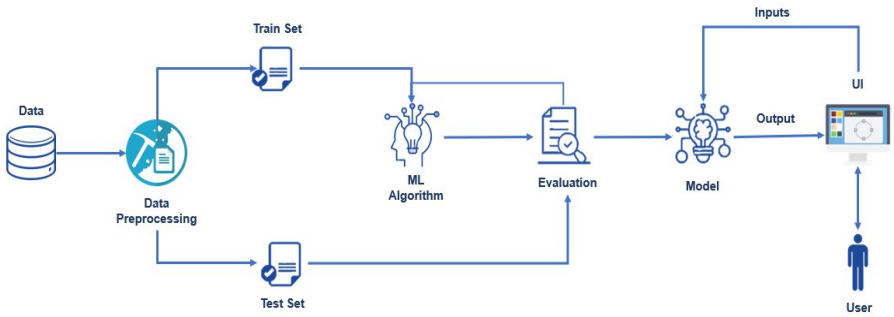
Date	13th February 2026
Team ID	LTVIP2026TMIDS81934
Project Name	Electric Motor Temperature Prediction using Machine Learning
Maximum Marks	4 Marks

Technical Architecture:

The Deliverable shall include the architectural diagram as below and the information as per the table1 & table 2

Example: Order processing during pandemics for offline mode

Reference: https://developer.ibm.com/patterns/ai-powered-backend-system-for-order-processing-during-pandemics/



Guidelines:

Include all the processes (As an application logic / Technology Block)
Provide infrastructural demarcation (Local / Cloud)
Indicate external interfaces (third party API's etc.)
Indicate Data Storage components / services
Indicate interface to machine learning models (if applicable)

Table-1 : Components & Technologies:

S.No	Component	Description	Technology
1.	User Interface	Web-based UI for interaction: home page, manual input form, sensor simulation page.	HTML5, CSS3, Jinja2 (templating)
2.	Application Logic-1	Core web application – handles routing, request processing, and template rendering.	Python 3.9, Flask 2.3.3
3.	Application Logic-2	Data preprocessing – normalizes input features using MinMaxScaler.	Python, scikit-learn 1.3.0
4.	Application Logic-3	Prediction logic – loads the trained Random Forest model and generates temperature predictions.	Python, scikit-learn, joblib 1.3.2
5.	Database	No database used in the current version. Predictions are not persisted.	Not applicable
6.	Cloud Database	Not implemented (planned for future cloud deployment).	–
7.	File Storage	Stores the trained model (model.save), scaler (transform.save), and dataset (pmsm_temperature_data.csv).	Local filesystem
8.	External API-1	None (future scope may include IoT/MQTT integration).	–

9.	External API-2	None.	–
10.	Machine Learning Model	Random Forest Regressor trained on 1.33M records to predict permanent magnet temperature (pm).	scikit-learn (RandomForestRegressor), joblib
11.	Infrastructure (Server / Cloud)	Deployed locally on a development machine. Future deployment planned on IBM Cloud / AWS.	Local: Python environment (Flask built-in server). Cloud: (future) Cloud Foundry / Kubernetes.

Table-2: Application Characteristics:

S.No	Characteristics	Description	Technology / Justification
1.	Open-Source Frameworks	The project is built entirely using open-source technologies.	Flask, scikit-learn, pandas, numpy, matplotlib, seaborn, joblib, Jinja2
2.	Security Implementations	The current local deployment has no authentication or encryption. For production, security measures (user login, HTTPS, IAM) would be added.	None (local); future: OWASP guidelines, JWT, HTTPS
3.	Scalable Architecture	The solution follows a **4-layer modular architecture** (UI, Application, Processing, Data). Each layer can be scaled independently. For example, the Flask layer can be replicated behind a load balancer, and the model can be served via a dedicated ML service.	Modular layered design; future containerization (Docker) and orchestration (Kubernetes).
4.	Availability	Currently runs on a single machine – a single point of failure. High availability can be achieved by deploying on cloud platforms with auto-scaling and multi-zone replication.	Local: no redundancy. Future: cloud (IBM Cloud / AWS) with load balancers and multiple instances.
5.	Performance	Prediction response time is **< 2 seconds** for a single request. The model	Efficient Random Forest implementation; future: add

S.No	Characteristics	Description	Technology / Justification
		is optimized (50 trees) and input scaling is lightweight. No caching implemented yet.	Redis cache for repeated inputs, use CDN for static files.

References:

https://c4model.com/

https://developer.ibm.com/patterns/online-order-processing-system-during-pandemic/

https://www.ibm.com/cloud/architecture

https://aws.amazon.com/architecture

https://medium.com/the-internal-startup/how-to-draw-useful-technical-architecture-diagrams-2d20c9fda90d